# Beispiel für verteilte Systeme



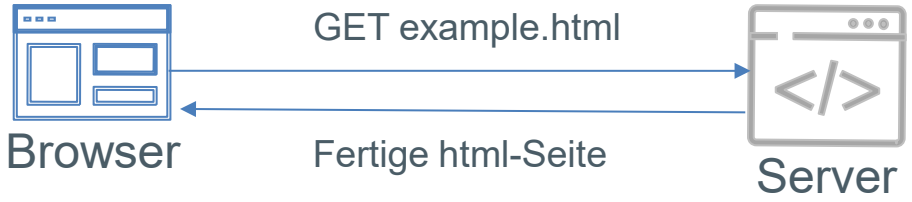

Zum Beispiel:
- eCampus
- Moodle
- …

Die grünen Pfeile sind Nachrichten, die über ein Rechnernetz verschickt werden!

# Server-Side Rendering und Client-Side Rendering

Für die Produktion der HTML-Seiten gibt es im Prinzip zwei Herangehensweisen:

- Server-Side Rendering
  Der Server liefert dem Client eine fertige HTML-Seite, der Client muss sie nur noch darstellen („rendern“) – Beispiel: JavaServerPages (~2000).




- Client-Side Rendering
  Der Server liefert dem Client eine fast leere HTML-Seite (siehe links) und JavaScript. Die Scripts holen Daten von Servern und rendern die Seite.

Vor- und Nachteile?

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>CSR</title>
</head>
<body>
<div id="root"><!-- blank --></div>
<script src="/bundle.js"></script>
</body>
</html
```

# Beispiele für verteilte Systeme: Netflix



Jeder grüne Punkt symbolisiert einen Micro-Service und kann drei bis mehrere hundert Male installiert sein.

Die blauen Linien sind die Aufrufwege. Alle Komponenten sind auf mehreren zehntausend Amazon Cloud-Servern deployed.

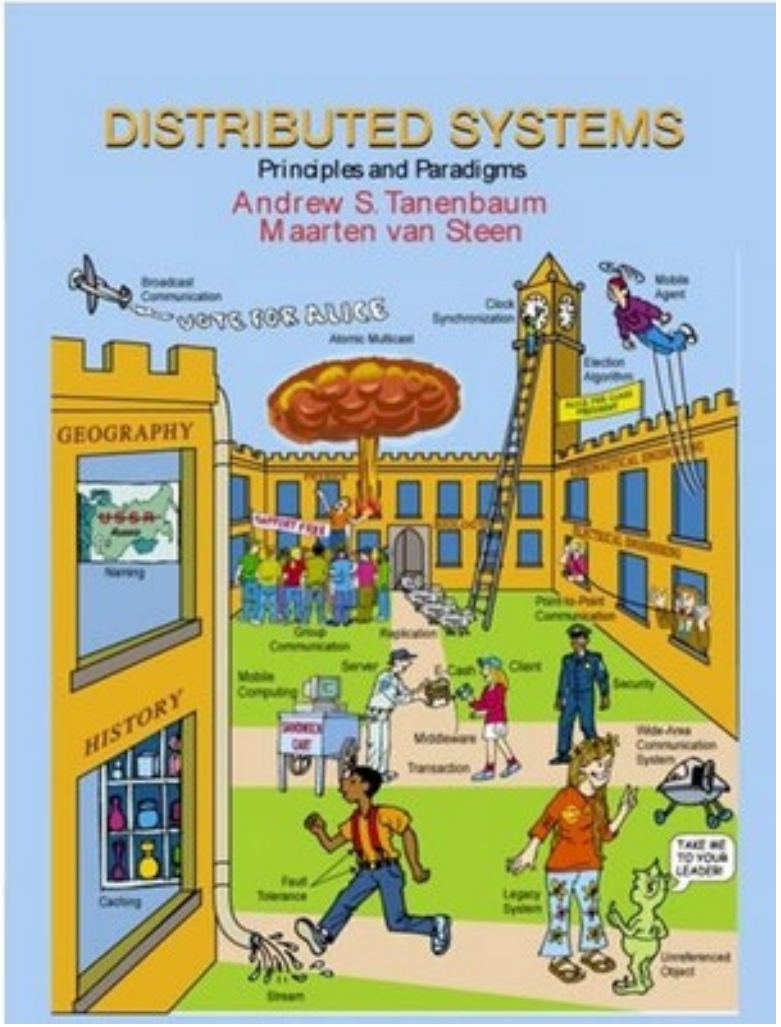
Quelle:
https://www.embarc.de/netflix-architektur-blogserie-teil-1

# Was ist ein verteiltes System?

## Definitionen:

*(1) A **distributed system** is a collection of independent computers that appears to its users as a single system.*

*(2) Many real systems consist of a number of physically distributed components that work independently using their private storage, but also communicate from time to time by explicit message passing. Such systems are called **distributed systems**.*




Quellen:
1) M. v. Steen, A. Tanenbaum, : Distributed Systems: Principles and Paradigms, Pearson.
2) K. Apt, F. de Boer, E. Olderog: Verification of sequential and concurrent programs, Springer Verlag, 2010

# Ziele verteilter Systeme



**Zugriff auf Ressourcen**

Zum Beispiel: Zugriff auf Dateien, Drucker, Web Seiten, Computer, …

**Offenheit**

**Interoperabilität** (Fähigkeit mit Komponenten andere Hersteller zusammenzuarbeiten) und **Erweiterbarkeit**

**Transparenz**

**Unterschiedliche** Hardware, Betriebssysteme, …; **Physischer Ort** einer Ressource; **Migration der HW; Umzug** des Nutzers; **Replikation der Daten, Parallelität, Fehlertransparenz**

**Skalierbarkeit**

Unter Skalierbarkeit versteht man die Fähigkeit eines Systems, Netzwerks oder Prozesses zur Größenveränderung

# Abstimmung

- Sind die folgenden Technologien „offen“ ?
  - HTTP
  - Windows PowerShell
  - TCP

- Wie kann man am besten ein System skalieren?
  1. Mehr Server kaufen
  2. Einen größeren Server kaufen
  3. Last zwischen Browser und Server verteilen
  4. Alle davon
  5. Kommt drauf an
  6. Keines davon

# Pitfalls („Fallgruben“) – Fallacies („Unwahrheiten“)

Peter Deutsch, Sun Microsystems, nannte 7 grundlegende und wiederkehrende Fehler beim Design von verteilten Systemen, James Gosling („the father of Java“) fügte Nummer 8 hinzu.

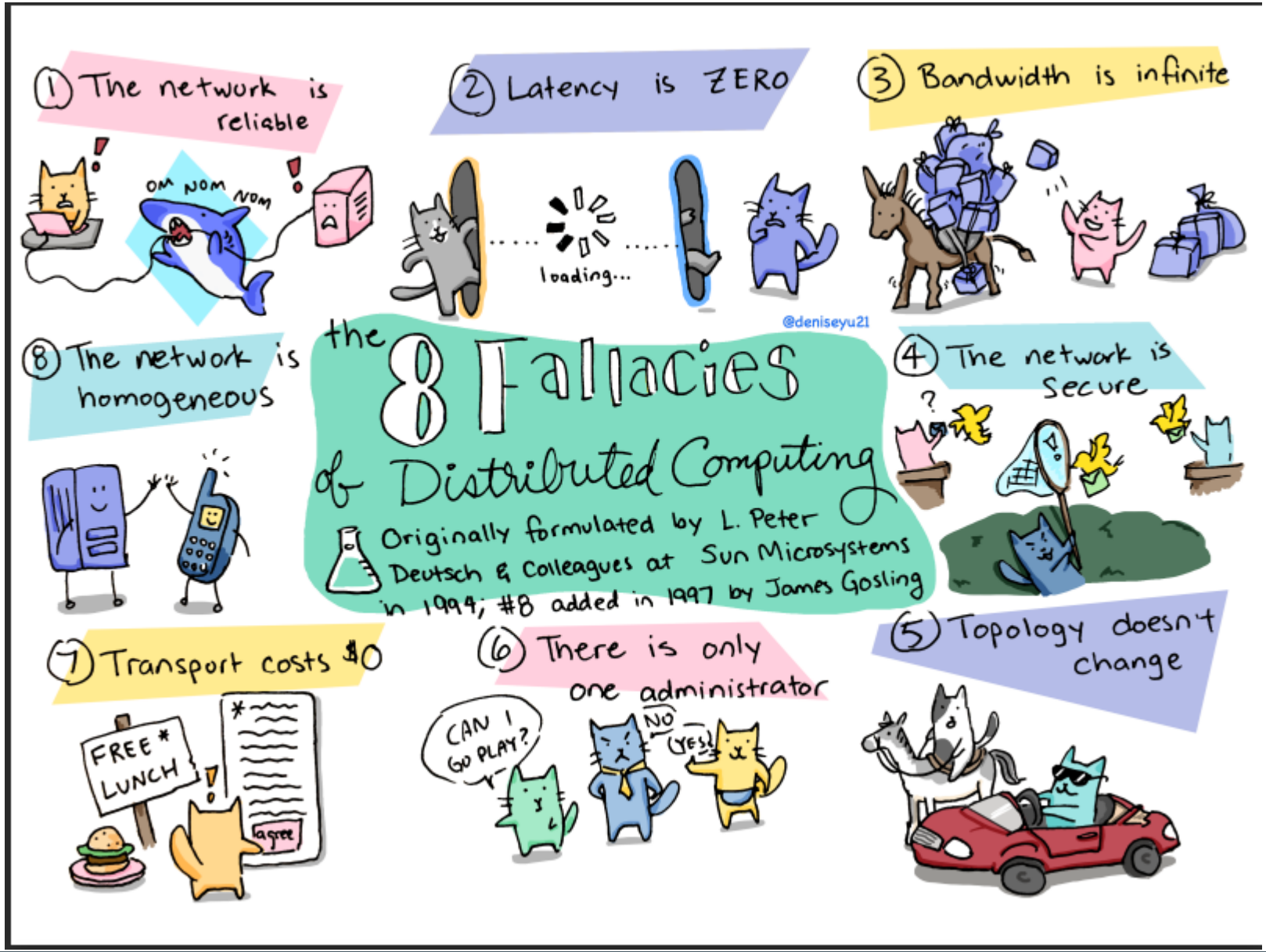


1. Das Netzwerk ist verlässlich
2. Das Netzwerk ist sicher
3. Das Netzwerk ist homogen
4. Die Topologie ändert sich nicht
5. Latenzzeit ist Null
6. Bandbreite ist unendlich
7. Transportkosten sind Null
8. Es gibt einen Administrator



# Pitfalls

Bilden Sie kleine Gruppen (ca. 4 Personen), suchen Sie sich eine der 8 Pitfalls aus und schreiben Sie auf, warum diese wirklich eine „Fallgrube“ ist.




Quelle: https://deniseyu.io/distsystalk

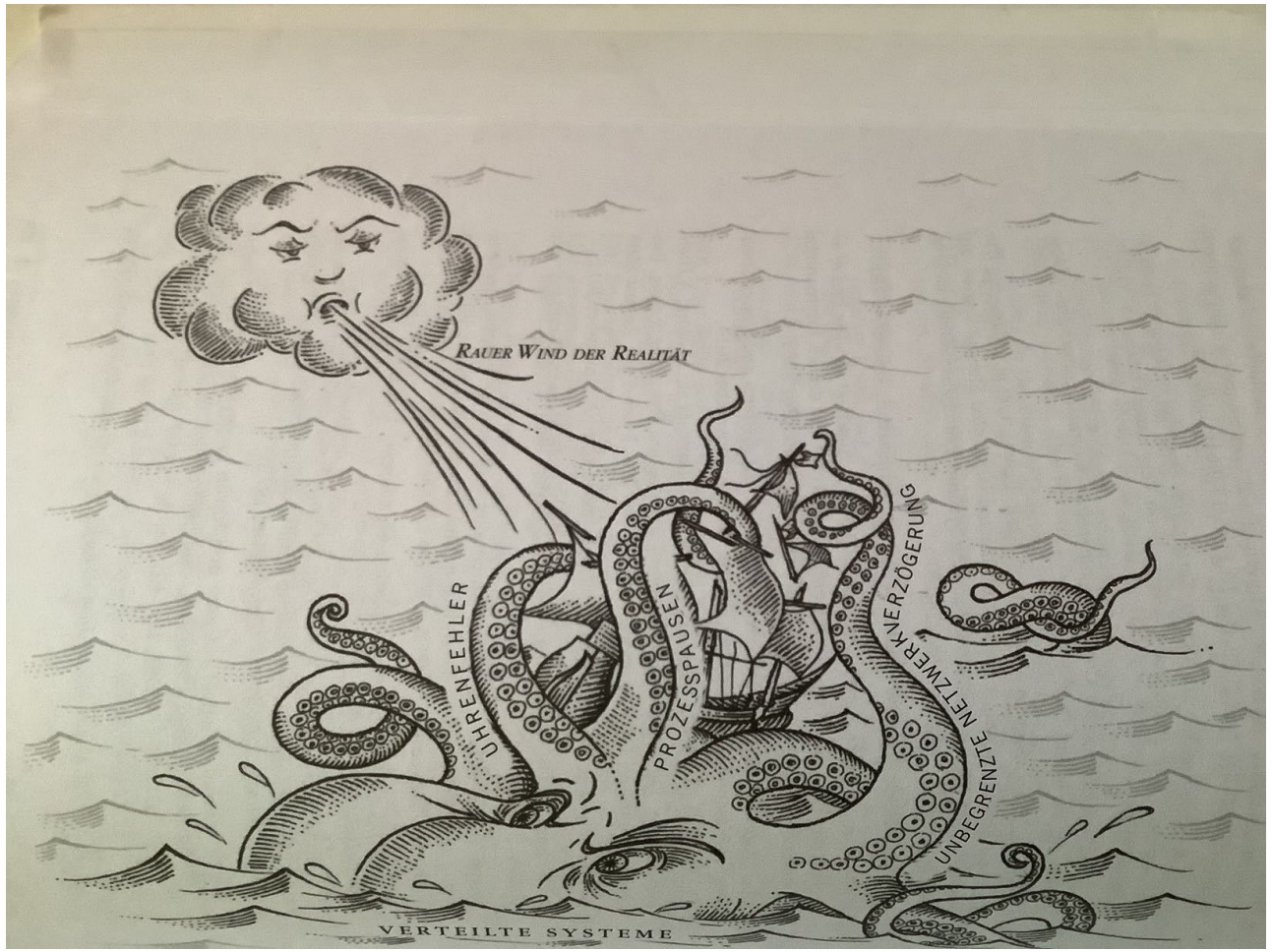
# Welche Themen beschäftigen uns bei verteilten Systemen?

1. Ein „System“ hat eine gewisse Funktionalität.
   Wie „verteilt“ man diese?
   Hier werden wir einige grundlegende „Architekturen“ besprechen.

2. Wie kann ein Teil des verteilten Systems auf Funktionen eines anderen Teils zugreifen?
   Wir haben in PI 1 gelernt, wie man z.B. mittels fetch auf Services auf anderen Servern zugreift. Es gibt aber mehr Möglichkeiten …

3. Wie gehen wir mit den Pitfalls um?
   Es gibt viele grundlegende Algorithmen zur Koordination und zur Sicherstellung der Konsistenz. Diese behandeln wir erst in der Master-Vorlesung „Verteilte Systeme“. Ansonsten: Immer den schlimmsten Fall annehmen (und testen!).

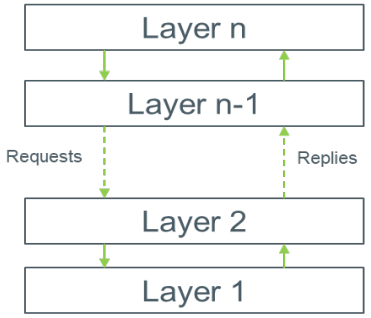
# Die Herausforderungen verteilter Systeme



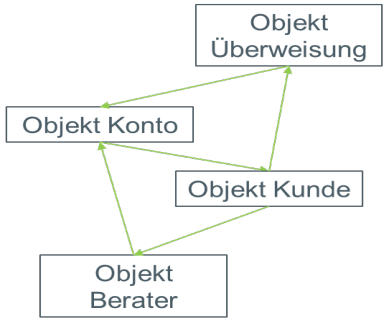

Quelle: Martin Kleppman: Designing data-intensive applications, O'Reilly

# Vier grundlegende Architekturparadigmen für verteilte Systeme

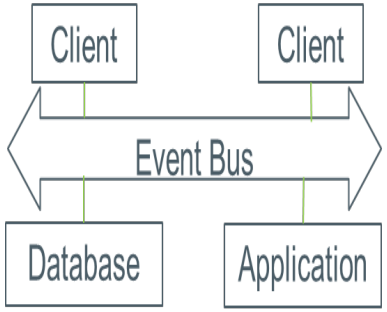
Layered Architecture



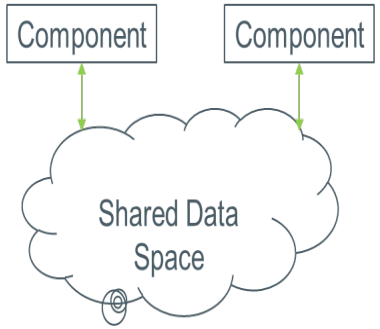

Objekt-basierte Architektur




Event-Driven Architecture



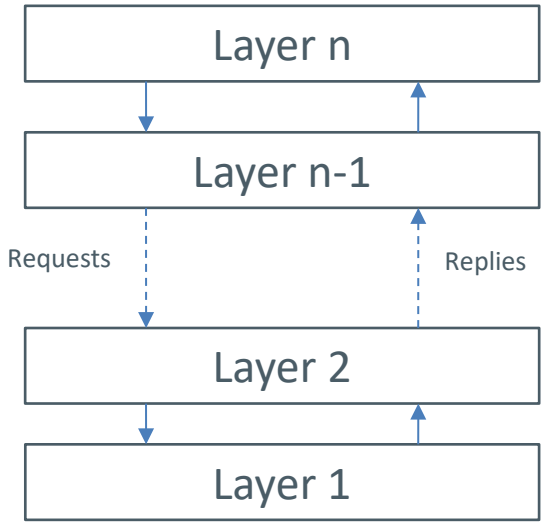

Shared data-space



# Layered Architecture

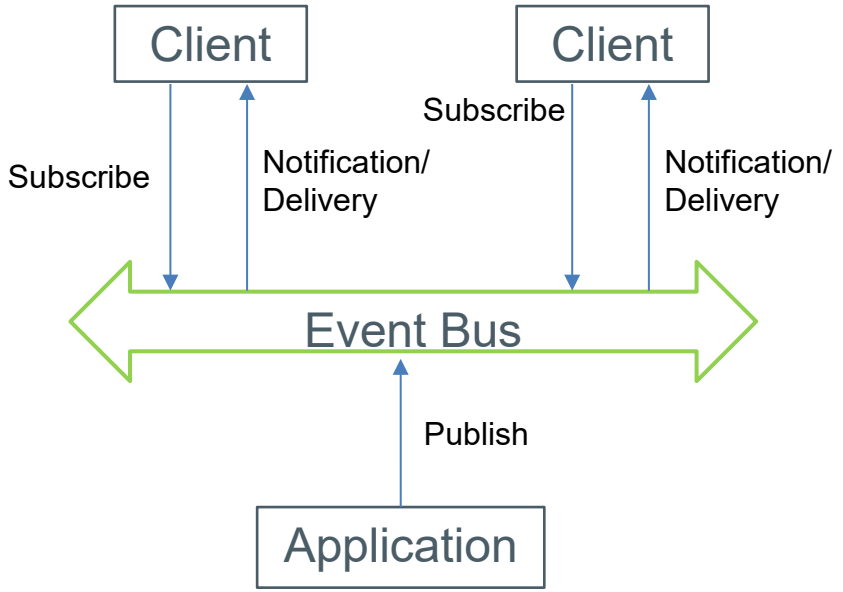
- Komponenten auf Schicht k können Komponenten der Schicht k-1 aufrufen
- Klare Aufteilung der Verantwortlichkeiten (z.B. Datenhaltung auf Schicht 1, Benutzer-Interaktion auf Schicht N)

- Klassisches Beispiel: ISO/OSI-Model
- In Software-Architektur:
  - Präsentation (UI)
  - Geschäftslogik
  - Datenzugriffsschicht
  - Datenbank/Persistenz



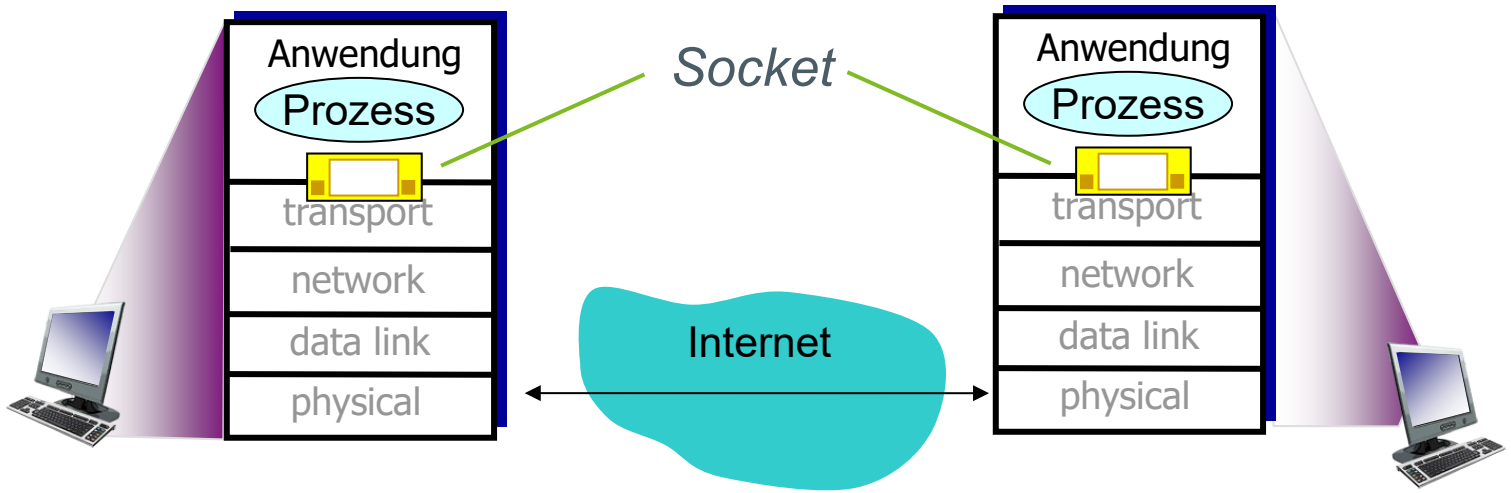
# Event-Based Architecture

- „Ereignisse“ lösen Kommunikation aus (z.B. Neuer Kunden angelegt, neue Bestellung eingetroffen, …).
- Komponenten reagieren auf Ereignisse und stoßen ggf. weitere Kommunikation an.
- Publisher veröffentlichen (**pub**lish) Nachrichten auf einem Kanal
- Clients abonnieren (**sub**scribe) einen Kanal und werden benachrichtigt, wenn eine neue Nachricht vorhanden ist
- Flexibel erweiterbar durch lose Kopplung, gute Skalierbarkeit.
- Event-basierte Systeme sind schwer zu debuggen.
- Beispiele:
  - Smart-Home-Systeme
  - Social Media Feeds
  - Push-Benachrichtigungen auf Mobilegeräten



# Recap: Wie senden und empfangen Prozesse die Nachrichten?

- Prozesse senden und empfangen Nachrichten von Sockets (“Steckdosen”)
  - Sendender Prozess schiebt die Nachricht in den Socket
  - und verlässt sich auf die Transport-Infrastruktur, um die Nachricht zum anderen Socket zu liefern, wo sie vom empfangenden Prozess entgegengenommen wird



# Synchrone Kommunikation

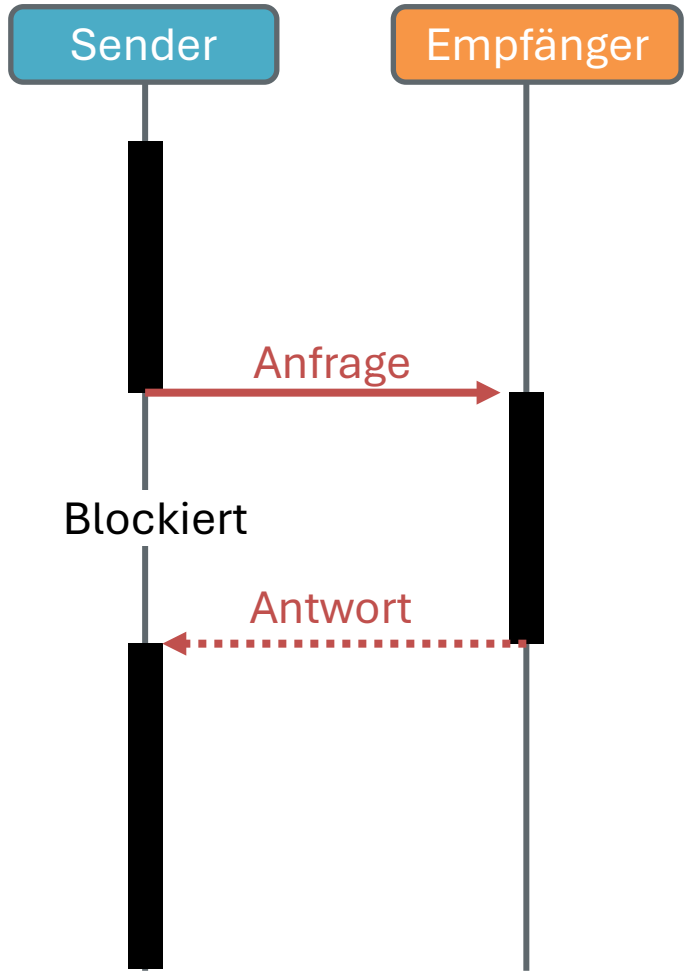
Der **Sender** schickt eine Nachricht und **wartet** bis der **Empfänger** die Nachricht beantwortet. Für das Kommunikationsmuster müssen Sender und Empfänger *gleichzeitig* aktiv sein.

→Analogie: Fragen per Telefon

**Beispiel:** Abrufen von Daten aus einer REST-API oder Datenbank

Vorteile?

Nachteile?



# Asynchrone Kommunikation

Der **Sender** schickt eine Anfrage und **wartet nicht** auf eine Antwort.
Der **Empfänger** kann die Nachricht verarbeiten und antworten, sobald er dazu in der Lage ist.

→ Analogie: Fragen per E-Mail oder WhatsApp

**Beispiel:** Anfragen mit fetch(url).then(callback)

☺ Höhere Effizienz, da Sender nicht blockiert wird

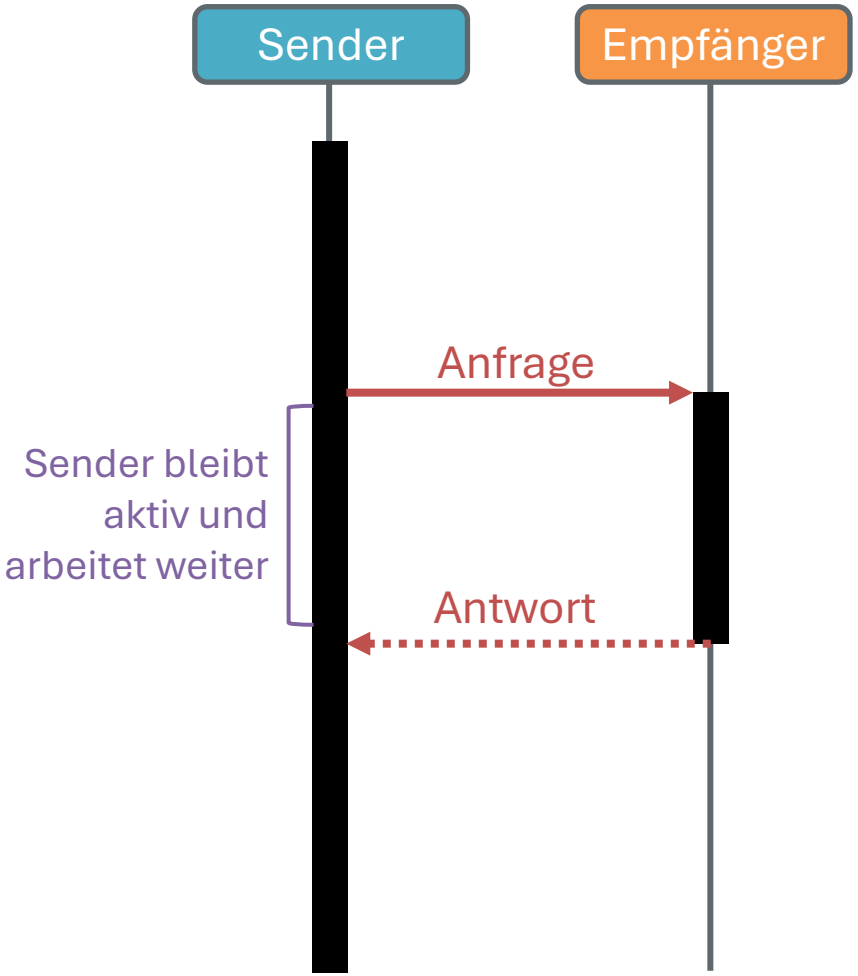
☺ Reduziert Abhängigkeit vom Server (z.B. Ausfallsicherheit)

☺ Vereinfacht Lastverteilung und Parallelisierung auf Empfängerseite

☹ Komplexere Fehlerbehandlung

☹ Reihenfolge nicht garantiert. Warum?

☹ Debuggen sehr schwer



# Synchron? Wirklich?

